| Logic | Description |
|---|---|
| `RoundRobinRouting` | The first received message goes to the first routee, the next message goes to the second routee, and so on. When all routees have received a message, everything starts again from the beginning. |
| `RandomRouting` | Each received message goes to a randomly selected routee. |
| `ConsistentHashing` | A consistent hashing is applied to the message to select a routee. This algorithm is especially useful for distributing the work evenly among routees whose number may change over time. |
| `withBroadcastPredicate` | A message is sent to all routees that satisfy a predicate `function T => Boolean`, where T is the type of message sent. |

In sections 6.3.2 and 6.3.4, you learn more about the options and how to use them. These options are used in pool routers and group routers.

## *6.3.1 Akka pool router*

The pool router creates and manages routees for you. You do not have to explicitly create the actors to which it forwards messages. A pool can be used if all routees exist only in the router's domain and no further interaction with them is expected. For simple routees, a pool is a good choice.

To define a pool, you specify the number of instances and the behavior the pool needs to instantiate. Then you `.spawn` the pool like any other actor. It is an `ActorRef` you can send messages to, as you can see in the following example (note that the type of the router must be the same as its routees):